

Reviewer Pack — Critical Simplices in Discrete Morse Theory Bound Tseitin Re...
Entry c360d324fb42 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler
2026-05-20 02:01:02 UTC

Critical Simplices in Discrete Morse Theory Bound Tseitin Resolution Length

Entry ID: c360d324fb42 Verdict: INCONCLUSIVE Recorded: 2026-05-20 02:01:02 UTC

1. Conjecture

Field A (mathematical branch): Discrete Morse Theory Field B (complexity object): Resolution Length of Tseitin Formulas

Statement:

For a Tseitin formula on a d -regular graph G with n vertices, the resolution length is $\geq 2^{\{\Omega(\nu(G))\}}$, where $\nu(G)$ is the minimum number of critical simplices in any discrete Morse function on G 's simplicial complex. This holds for all graphs with expansion $\lambda(G) \geq \Omega(\sqrt{\log n})$.

Rationale (proposer's reasoning):

Discrete Morse theory captures topological simplifications of simplicial complexes, which may correlate with the combinatorial complexity of resolution proofs. The critical simplices could encode structural bottlenecks in the proof system, while expansion ensures the graph's connectivity sustains the lower bound.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 84d523f8b583328b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (d * n) % 2 != 0 or d < 1 or n < 1:
            return None
        adj_list = {i: [] for i in range(n)}
        edges_used = set()
        for _ in range(d * n // 2):
            u, v = random.sample(range(n), 2)
            if (u, v) not in edges_used and (v, u) not in edges_used:
                adj_list[u].append(v)
                adj_list[v].append(u)
                edges_used.add((u, v))
        return adj_list

    def compute_expansion(graph):
        n = len(graph)
        degrees = [len(neighbors) for neighbors in graph.values()]
        max_degree = max(degrees)
        min_degree = min(degrees)
        expansion = (max_degree + min_degree) / 2
        return expansion

    def persistent_homology_based_morse_matching(graph):
        # Simplified Morse matching algorithm using a greedy approach
        n = len(graph)
        critical_simplices = []
        visited = set()

        for node in range(n):
            if node not in visited:
                stack = [node]
                while stack:
                    current = stack.pop()
```

```

        if current not in visited:
            visited.add(current)
            neighbors = graph[current]
            if len(neighbors) == 1:
                critical_simplices.append((current, neighbors[0]))
            else:
                for neighbor in neighbors:
                    if neighbor not in visited:
                        stack.append(neighbor)
    return len(critical_simplices)

def resolution_length(graph):
    # Simplified DPLL-based proof size estimator
    n = len(graph)
    clauses = []
    for node in range(n):
        clause = [node, -node]
        clauses.append(clause)

    def dpll(clauses, assignment, model):
        if not clauses:
            return True
        literal = find_pure_literal(clauses)
        if literal is None:
            literal = choose_branching_literal(clauses)
        if literal > 0:
            new_assignment = assignment | {literal: True}
            new_model = model + [literal]
        else:
            new_assignment = assignment | {-literal: False}
            new_model = model + [-literal]

        if dpll([c for c in clauses if not unit_propagate(c, literal)], new_assignment, new_model):
            return True
        return False

    def find_pure_literal(clauses):
        pure_literals = set()
        for clause in clauses:
            positive = [l for l in clause if l > 0]
            negative = [-l for l in clause if l < 0]
            if len(positive) == 1 and all(-p not in pure_literals for p in positive):
                pure_literals.add(positive[0])
            if len(negative) == 1 and all(p not in pure_literals for p in negative):
                pure_literals.add(negative[0])
        return None

    def choose_branching_literal(clauses):
        literals = set()
        for clause in clauses:
            literals.update([l for l in clause if l > 0] + [-l for l in clause if l < 0])
        return random.choice(list(literals))

    assignment = {}
    model = []
    return dpll(clauses, assignment, model)

```

```

n = random.randint(5, 40)
d = random.randint(2, min(n - 1, 3))
graph = generate_d_regular_graph(n, d)
if not graph:
    return {
        "metric_name": "resolution_length",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

expansion = compute_expansion(graph)
if expansion < 0.1 * math.sqrt(math.log(n)):
    return {
        "metric_name": "resolution_length",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "expansion_too_low"
    }

nu_G = persistent_homology_based_morse_matching(graph)
resolution_len = resolution_length(graph)

if resolution_len is not None:
    c = 0.2
    conjecture_holds = resolution_len >= 2 ** (c * nu_G)
else:
    return {
        "metric_name": "resolution_length",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "resolution_length_not_computed"
    }

return {
    "metric_name": "resolution_length",
    "metric_value": resolution_len,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 9973) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

```

```

total_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None)
num_trials = len(results)
mean_metric_value = total_metric_value / num_trials if num_trials > 0 else 0
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / num_trials)

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    result = "SUPPORTED"
elif support_fraction >= 0.8:
    result = "SUPPORTED"
else:
    counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"]), None)
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    result = f"FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}"

print(f"RESULT: {result} mean={mean_metric_value:.2f} std={std_metric_value:.2f} support_fraction={support_fraction:.2f}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'resolution_length', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffce68a8.py", line 161, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffce68a8.py", line 133, in run_trial
    resolution_len = resolution_length(graph)
                   ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffce68a8.py", line 108, in resolution_length
    return dpll(clauses, assignment, model)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffce68a8.py", line 85, in dpll
    if dpll([c for c in clauses if not unit_propagate(c, literal)], new_assignment, new_model):
           ~~~~~
NameError: name 'unit_propagate' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined 'unit_propagate' function, preventing resolution length calculation | next: Implement missing unit propagation logic in the DPLL solver

11. Audit log (LLM calls)

Total LLM calls: 14

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	138474
2	propose	ollama_remote	qwen3:8b	0	0	66731
3	propose	ollama_remote	qwen3:8b	0	0	96615
4	propose	ollama_remote	qwen3:8b	0	0	72327
5	propose	ollama_remote	qwen3:8b	0	0	140400
6	propose	ollama_remote	qwen3:8b	0	0	65535
7	preregistration	ollama_remote	qwen3:8b	0	0	31641
8	novelty	ollama_remote	qwen3:8b	0	0	27500
9	novelty	ollama_remote	qwen3:8b	0	0	22230
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15607
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14003
12	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8188
13	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16987
14	judge	ollama_remote	qwen3:8b	0	0	21834

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 738073 ms total latency. Provider mix: {'ollama_remote': 14}

(full prompt+response transcripts available in `research/audit/c360d324fb42.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c360d324fb42.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c360d324fb42.tar.gz` (if generated)